```
import serial
import serial.tools.list_ports
import numpy as np
from scipy.signal import butter, iirnotch, filtfilt
import matplotlib.pyplot as plt
import time

# === CONFIGURATION ===
SAMPLE_COUNT = 1000      # Number of data points
FS = 100                 # Sampling frequency (Hz)
THRESHOLD = 2.0          # Voltage danger threshold
MOVING_AVG_WINDOW = 50   # Smoothing window
HP_CUTOFF = 10           # High-pass filter cutoff (Hz)
NOTCH_FREQ = 50          # Notch frequency (Hz)

# === FILTER FUNCTIONS ===
def apply_notch_filter(data, fs, freq=50, Q=35):
    wo = freq / (fs / 2)
    bw = wo / Q
    b, a = iirnotch(wo, bw)
    return filtfilt(b, a, data)

def apply_highpass_filter(data, fs, cutoff=10):
    b, a = butter(2, cutoff / (fs / 2), btype='high')
    return filtfilt(b, a, data)

def moving_average(data, window_size=50):
    return np.convolve(data, np.ones(window_size)/window_size, mode='same')

# === MAIN FUNCTION ===
def main():
    # Step 1: List and choose COM port
    ports = list(serial.tools.list_ports.comports())
    if not ports:
        print("❌ No serial ports found!")
        return
    print("Available serial ports:")
    for p in ports:
        print(f"  {p.device} - {p.description}")
    port_name = input("Enter COM port (e.g., COM3): ").strip()
    baud = int(input("Enter baud rate (e.g., 9600): ").strip())

    try:
        ser = serial.Serial(port_name, baud, timeout=1)
        time.sleep(2)
    except Exception as e:
        print(f"❌ Could not open port {port_name}: {e}")
        return

    print(f"\n☑ Connected to {port_name} @ {baud} baud")
```

```
    print(f"📡 Collecting {SAMPLE_COUNT} samples...\n")

    # Step 2: Read and debug data
    raw_data = []
    while len(raw_data) < SAMPLE_COUNT:
        try:
            line = ser.readline().decode('utf-8', errors='ignore').strip()
            print(f"[DEBUG] Received line: {repr(line)}")  # ← SHOW RAW DATA

            if not line:
                continue
            try:
                value = float(line)
                raw_data.append(value)
                status = "⚠️ DANGER!" if value > THRESHOLD else "☑ ENGINE OK"
                print(f"Raw: {value:.3f} V → {status}")
            except ValueError:
                print(f"[WARNING] Skipping invalid data: {repr(line)}")
        except Exception as e:
            print(f"[ERROR] Serial read error: {e}")
            break

    ser.close()
    print("\n🔌 Serial port closed.")

    if not raw_data:
        print("❌ No valid data received.")
        return

    # Step 3: Processing
    raw_array = np.array(raw_data)
    filtered = apply_notch_filter(raw_array, FS, NOTCH_FREQ)
    filtered = apply_highpass_filter(filtered, FS, HP_CUTOFF)
    smoothed = moving_average(filtered, MOVING_AVG_WINDOW)

    t = np.arange(len(raw_array)) / FS
    threshold_line = np.full_like(t, THRESHOLD)

    # Step 4: Plotting
    plt.figure(figsize=(12, 6))
    plt.plot(t, raw_array, label="Raw Signal", alpha=0.6, color='blue')
    plt.plot(t, smoothed, label="Filtered + Smoothed", linewidth=2, color='green')
    plt.plot(t, threshold_line, '--', label=f"Threshold = {THRESHOLD} V",
color='red')

    plt.title("FPGA Signal Monitoring and Filtering")
    plt.xlabel("Time (s)")
    plt.ylabel("Voltage (V)")
    plt.legend()
    plt.grid(True)
```

```
    plt.tight_layout()
    plt.show()

# Run the program
if __name__ == "__main__":
    main()
```